

Customer Segmentation and Pricing Optimization

Anant

Running Code

Running RFM based customer segmentation and pricing optimization on the dataset 'Online Retail II' available in the UCI ML Repository.

```
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```
library(lubridate)
```

Attaching package: 'lubridate'

The following objects are masked from 'package:base':

date, intersect, setdiff, union

```
library(ggplot2)
library(factoextra)
```

Welcome to factoextra!

Want to learn more? See two factoextra-related books at <https://www.datanovia.com/en/product-guide-to-principal-component-methods-in-r/>

```
library(dplyr)
library(purrr)
library(scales)
```

Attaching package: 'scales'

The following object is masked from 'package:purrr':

discard

```
df <- read.csv("online_retail_II.csv", stringsAsFactors = FALSE)
dim(df)
```

```
[1] 1067371      8
```

```
colnames(df) <- make.names(colnames(df))

#cleaning
#date
df$InvoiceDate <- as.POSIXct(df$InvoiceDate, format="%Y-%m-%d %H:%M:%S")
df <- df %>% filter(!is.na(Customer.ID))

#remove cancellations (Invoice starting with C)
df <- df %>% filter(!grepl("^C", Invoice))

#Remove invalid values
df <- df %>% filter(Quantity > 0, Price > 0)

#creating total price
df <- df %>% mutate(TotalPrice = Quantity * Price)
```

```

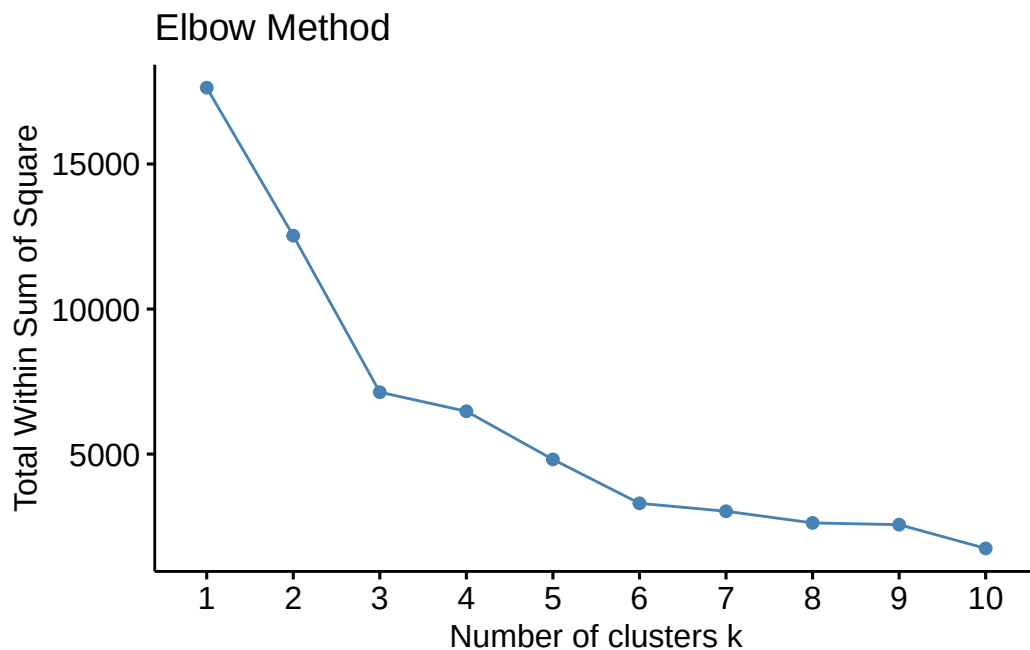
#snapshot date , 'today'
snapshot_date <- max(df$InvoiceDate) + 1

#RFM calculation
rfm <- df %>%
  group_by(Customer.ID) %>%
  summarise(
    Recency = as.numeric(snapshot_date - max(InvoiceDate)),
    Frequency = n_distinct(Invoice),
    Monetary = sum(TotalPrice)
  )

#scaling
rfm_scaled <- scale(rfm[, -1])

fviz_nbclust(rfm_scaled, kmeans, method = "wss") +
  ggtitle("Elbow Method")

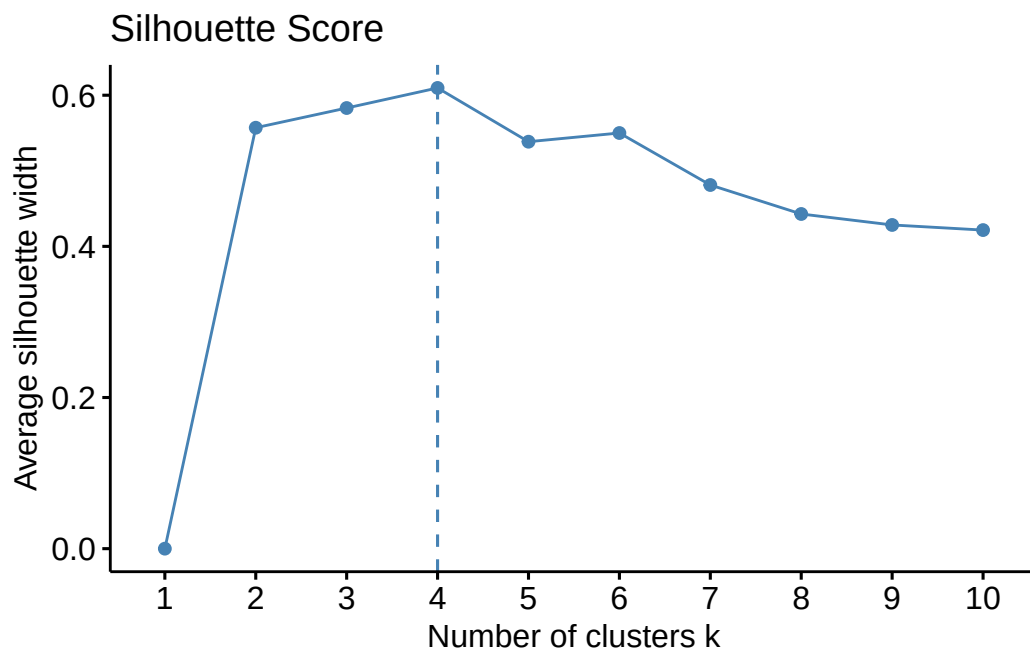
```



```

fviz_nbclust(rfm_scaled, kmeans, method = "silhouette") +
  ggtitle("Silhouette Score")

```



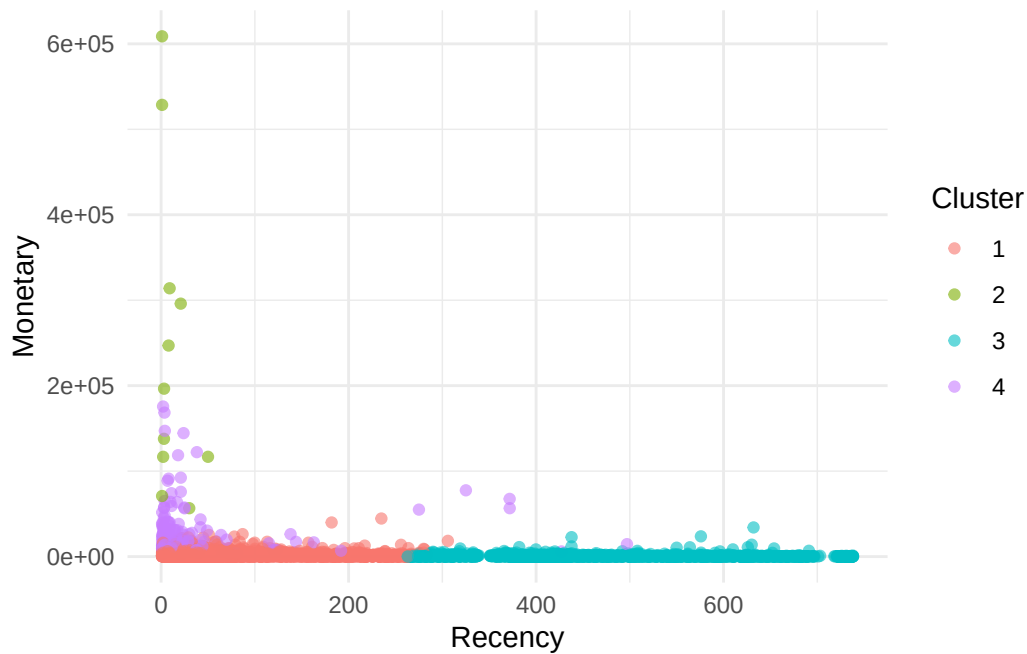
```
set.seed(123)
kmeans_model <- kmeans(rfm_scaled, centers = 4, nstart = 25)

rfm$Cluster <- as.factor(kmeans_model$cluster)
# Cluster summary
rfm %>%
  group_by(Cluster) %>%
  summarise(
    Avg_Recency = mean(Recency),
    Avg_Frequency = mean(Frequency),
    Avg_Monetary = mean(Monetary),
    Count = n()
  )
```

```
# A tibble: 4 x 5
  Cluster Avg_Recency Avg_Frequency Avg_Monetary Count
  <fct>    <dbl>         <dbl>         <dbl> <int>
1 1       71.9          5.85          2222.  3696
2 2       11.8         194.          244510.   11
3 3       468.          2.16           753.  1946
4 4       27.7         40.1          23891.   225
```

```
#View(df)
#dim(df)

#plotting rfm
ggplot(rfm, aes(x = Recency, y = Monetary, color = Cluster)) +
  geom_point(alpha = 0.6) +
  theme_minimal()
```



```
aggregate(cbind(Recency, Frequency, Monetary) ~ Cluster, data = rfm, mean)
```

	Cluster	Recency	Frequency	Monetary
1	1	71.88208	5.848755	2221.895
2	2	11.76395	193.545455	244510.006
3	3	467.72725	2.162384	753.414
4	4	27.70966	40.066667	23891.336

```
df <- merge(df, rfm[, c("Customer.ID", "Cluster")], by = "Customer.ID")

segment_revenue <- df %>%
  group_by(Cluster) %>%
  summarise(Revenue = sum(TotalPrice))
```

```
#estimating price elasticity (base)
df_model <- df %>%
  filter(Quantity > 0, Price > 0) %>%
  mutate(
    logQ = log(Quantity),
    logP = log(Price)
  )

model <- lm(logQ ~ logP, data = df_model)
summary(model)
```

Call:

```
lm(formula = logQ ~ logP, data = df_model)
```

Residuals:

Min	1Q	Median	3Q	Max
-5.8623	-0.8077	0.1120	0.6811	9.7080

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	2.003253	0.001525	1313.5	<2e-16 ***
logP	-0.558648	0.001344	-415.6	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.102 on 805547 degrees of freedom

Multiple R-squared: 0.1766, Adjusted R-squared: 0.1766

F-statistic: 1.727e+05 on 1 and 805547 DF, p-value: < 2.2e-16

```
#estimating price elasticity by cluster
elasticity_results <- df_model %>%
  group_by(Cluster) %>%
  summarise(
    model = list(lm(logQ ~ logP, data = pick(everything())))
  ) %>%
  mutate(
    beta = map_dbl(model, ~ coef(.x)[2])
  )
```

elasticity_results

```
# A tibble: 4 x 3
  Cluster model    beta
  <fct>    <list> <dbl>
1 1      <lm> -0.551
2 2      <lm> -0.515
3 3      <lm> -0.592
4 4      <lm> -0.587
```

```
#cannot run segmentation by product. hits memory limit
```

```
#setting changes in prices of cluster 1 and 2, where P1 is
#price for Cluster 1, P2 is price for Cluster 2
```

```
#since  $|E| < 1$ , we can increase prices,
#for exmaple, let's take a 2% increase for P1 and 5% for P2
delP1 <- 0.02
delP2 <- 0.05
df <- df %>%
  mutate(
    PriceChange = case_when(
      Cluster == 1 ~ delP1,
      Cluster == 2 ~ delP2,
      TRUE ~ 0
    ),
    NewPrice = Price * (1 + PriceChange),
    NewRevenue = Quantity * NewPrice
  )

original_revenue <- sum(df$TotalPrice)
new_revenue <- sum(df$NewRevenue)

change <- (new_revenue - original_revenue) / original_revenue
change
```

```
[1] 0.0168357
```

```
#implementing price elasticity
ped1 <- -0.55
```

```

ped2 <- -0.515
df <- df %>%
  mutate(
    AdjustedQuantity = case_when(
      Cluster == 2 ~ Quantity * (1 + ped2*delP2),
      Cluster == 1 ~ Quantity * (1 + ped1*delP1),
      TRUE ~ Quantity
    ),
    NewRevenue = AdjustedQuantity * NewPrice
  )
original_revenue <- sum(df$TotalPrice)
new_revenue <- sum(df$NewRevenue)

change <- (new_revenue - original_revenue) / original_revenue
change

```

```
[1] 0.007544349
```

```

ped1 <- -0.55
ped2 <- -0.515
price_changes <- seq(0, 0.30, by = 0.01) # 0% to 10%

results <- data.frame()

for (dp in price_changes) {
  #taking effective ped because elasticity will likely change with higher price)
  effective_ped1 <- ped1 - 2*dp
  effective_ped2 <- ped2 - 2*dp

  temp_df <- df %>%
    mutate(
      PriceChange = case_when(
        Cluster == 1 ~ dp,
        Cluster == 2 ~ dp,
        TRUE ~ 0
      ),
      NewPrice = Price * (1 + PriceChange),

      AdjustedQuantity = case_when(
        Cluster == 1 ~ Quantity * (1 + effective_ped1 * dp),

```



```

      Cluster == 2 ~ Quantity * (1 + effective_ped2 * dp),
      TRUE ~ Quantity
    ),

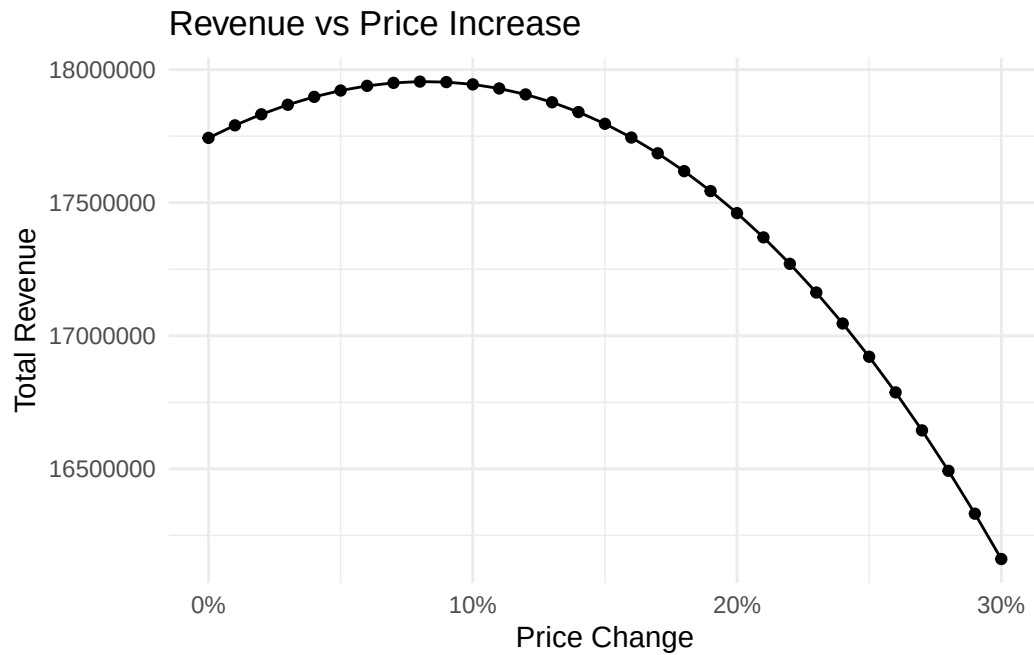
    NewRevenue = AdjustedQuantity * NewPrice
  )

new_rev <- sum(temp_df$NewRevenue)

results <- rbind(results, data.frame(
  PriceChange = dp,
  Revenue = new_rev
))
}

ggplot(results, aes(x = PriceChange, y = Revenue)) +
  geom_line() +
  geom_point() +
  theme_minimal() +
  scale_x_continuous(labels = percent) +
  labs(
    title = "Revenue vs Price Increase",
    x = "Price Change",
    y = "Total Revenue"
  )

```



```
optimal <- results[which.max(results$Revenue), ]  
optimal
```

	PriceChange	Revenue
9	0.08	17954945

Limitations

- No controls for product, time, or promotions
- Elasticity assumed constant in estimation
- Nonlinearity introduced via heuristic adjustment